

22. Autocomplete Suggestions (Typeahead)

Phase: L **Code:** `search_api/routers/category.py` (`/search/suggest` endpoint), `admin-ui/src/pages/SearchTest.jsx` (debounced query), `admin-ui/src/api.js` (`getSuggestions`)

What it is

Autocomplete (also called "typeahead" or "suggest-as-you-type") is the feature where a search box shows suggested completions as the user types. When you start typing "indust..." in our Admin UI search box, a dropdown appears with suggestions like "industrial pump", "industrial valve", "industrial automation" -- terms that exist in the database and are likely to produce good search results.

The feature has three parts working together:

1. **Backend suggest endpoint** (`GET /search/suggest`) -- queries the database for terms matching what the user has typed so far.
2. **Debounced API calls** -- the frontend waits 300ms after the user stops typing before making the API call, avoiding a flood of requests for every keystroke.
3. **HTML datalist element** -- a native browser dropdown mechanism that shows the suggestions below the input field.

The data source for suggestions is the `metadata->'add_words'` field in the `category_vectors` table -- these are enrichment keywords that the ETL pipeline generated for each category, stored as JSONB arrays. This means the suggestions come from actual content in the database, not from a separate autocomplete index.

Real-life analogy

Imagine you are at a library help desk. As you start spelling out what you are looking for on a notepad, the librarian glances at your writing:

Without debouncing: The librarian runs to the catalogue every time you write a single letter. You write "i" -- she runs. "in" -- she runs again. "ind" -- she runs

again. "indu" -- again. She is exhausted and most of the early trips were useless because "i" matches everything.

With debouncing (our approach): The librarian waits until you pause writing for about a third of a second. You write "i-n-d-u-s" rapidly, then pause to think. At that pause, she runs to the catalogue once and brings back suggestions for "indus...". Much more efficient -- she made one trip instead of five.

The datalist dropdown: Instead of reading suggestions aloud, she writes them on sticky notes and fans them out on the desk for you to pick from. This is what the HTML `<datalist>` element does -- it provides a native browser dropdown attached to the input field.

Why we used it here

User experience for the Admin UI search test page: The admin user testing search quality needs to formulate queries that match the vocabulary in the database. Without autocomplete, they would have to guess the exact terms. With autocomplete, they can start typing and see what terms the database actually contains, making it much faster to test different queries.

The suggest endpoint queries `metadata->'add_words'` -- the additional keywords generated by the AI enrichment step (LLM-generated synonyms and related terms). This is a good source for suggestions because:

1. These words are specifically designed to improve search recall -- they represent the vocabulary of the domain.
2. They exist in the metadata of actual indexed categories, so suggesting them guarantees at least some search results.
3. The distinct word list is much smaller than the full text corpus, making the query fast.

We use debouncing (300ms delay) because:

- Without it, the frontend would fire an API request on every keystroke. Typing "industrial pump" at normal speed (about 5 characters/second) would generate ~17 requests, of which only the last 1-2 are useful.
- 300ms is a standard debounce delay -- short enough that the user does not notice the lag, but long enough to skip intermediate keystrokes during continuous typing.

We use the native `<datalist>` HTML element instead of a custom dropdown component because:

- It requires zero CSS styling -- the browser renders it natively, consistent with OS conventions.
- It supports keyboard navigation, screen readers, and mobile interactions out of the box.
- It is sufficient for our use case (simple word suggestions, no images or metadata in the dropdown).

Code walkthrough

1. The suggest API endpoint -- `search_api/routers/category.py`

```
@router.get("/search/suggest")
def suggest(
    q: str = Query(..., min_length=2),      # at least 2 chars required
    limit: int = Query(10, ge=1, le=50),    # max 50 suggestions
):
    conn = get_connection()
    try:
        with conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute("""
                SELECT DISTINCT word
                FROM (
                    -- Unnest the JSONB array of add_words from each category
                    SELECT jsonb_array_elements_text(metadata->'add_words') AS word
                    FROM category_vectors
                    WHERE metadata->'add_words' IS NOT NULL
                ) sub
                -- Case-insensitive partial match: "pump" matches "pumping", "pump sys
                WHERE word ILIKE %s
                ORDER BY word
                LIMIT %s
            """, (f"%{q}%", limit))
            words = [r["word"] for r in cur.fetchall()]
    finally:
        conn.close()
    return {"suggestions": words}
```

Let's break down the SQL:

1. `jsonb_array_elements_text(metadata->'add_words')` -- this is a PostgreSQL function that "explodes" a JSONB array into individual rows. If a category's metadata has `{"add_words": ["pump", "valve", "motor"]}`, this produces 3 rows: "pump", "valve", "motor".

2. **SELECT DISTINCT word FROM (...) sub** -- wraps the explosion in a subquery and deduplicates. The same word might appear in multiple categories' `add_words` arrays.
3. **WHERE word ILIKE %s** -- `ILIKE` is PostgreSQL's case-insensitive `LIKE`. The `%` wildcards on both sides of `q` make it a substring match: `%pump%` matches "pump", "pumping", "hydraulic pump system", etc.
4. **ORDER BY word LIMIT %s** -- alphabetical order, capped at the requested limit.

Why `min_length=2` ? Single-character queries would match nearly every word in the database, returning useless suggestions and wasting a database query.

2. The client-side API function -- `admin-ui/src/api.js`

```
async function getSuggestions(q) {
  // Don't call the API for very short queries
  if (!q || q.length < 2) return [];

  // Build the URL with query parameters (properly encoded)
  const resp = await fetch(
    `${API_BASE}/search/suggest?q=${encodeURIComponent(q)}&limit=8`
  );

  // If the request fails (e.g., network error), return empty array silently
  // -- suggestions are a nice-to-have, not critical
  if (!resp.ok) return [];

  const data = await resp.json();
  return data.suggestions || [];
}
```

Note: `encodeURIComponent(q)` is important for URL safety. If the user types "C++ programming", the `+` characters need to be encoded as `%2B` in the URL, otherwise the server would interpret `+` as a space.

The limit is hardcoded to 8 in the client -- even though the server supports up to 50. Eight suggestions is a comfortable number for a dropdown without being overwhelming.

3. Debounced suggestions with `useRef` timer -- `admin-ui/src/pages/SearchTest.jsx`

```
const [suggestions, setSuggestions] = useState([]); // current suggestion list
const suggestTimer = useRef(null); // holds the debounce timeout
```

```

useEffect(() => {
  // Step 1: Cancel any pending suggestion request
  // (the user typed another character before the 300ms elapsed)
  clearTimeout(suggestTimer.current);

  // Step 2: If query is too short, clear suggestions and bail out
  if (query.length < 2) {
    setSuggestions([]);
    return;
  }

  // Step 3: Schedule a new suggestion request 300ms from now
  suggestTimer.current = setTimeout(async () => {
    const s = await getSuggestions(query);
    setSuggestions(s);
  }, 300);

  // Step 4: Cleanup -- if the component unmounts or query changes before
  // the timeout fires, cancel it to prevent a stale update
  return () => clearTimeout(suggestTimer.current);
}, [query]); // re-run this effect every time query changes

```

This is the **debounce pattern** using React's `useEffect` and `useRef` :

- **useRef** holds the timer ID across renders without triggering re-renders (unlike `useState`). We use it because we need to cancel the previous timer when the query changes, and we don't want the timer ID itself to cause a re-render.
- **clearTimeout(suggestTimer.current)** at the top cancels the previous timer. So if the user types "p", "pu", "pum", "pump" in quick succession (each keystroke re-runs the effect), only the last one survives the 300ms wait.
- **setTimeout(..., 300)** schedules the API call. 300ms is the industry-standard debounce delay -- Google Search, VS Code, and most search UIs use something between 200-400ms.
- **The cleanup function** (`return () => clearTimeout(...)`) is React's way of saying "if this effect re-runs or the component unmounts, run this cleanup first." It prevents the API call from firing after the component is gone (which would cause a React warning about updating unmounted components).

4. The HTML datalist element -- `admin-ui/src/pages/SearchTest.jsx`

```

{/* The search input references the datalist by its id */}


```

```

type="text"
list="query-suggestions"          // ← links to the <datalist> below
value={query}
onChange={(e) => setQuery(e.target.value)}
placeholder="e.g. ipari szivattyu gyarto"
autoComplete="off"                // disable browser's built-in autocomplete
/>

{/* The datalist holds the suggestion options */}
<datalist id="query-suggestions">
  {suggestions.map((s, i) => (
    <option key={i} value={s} />    // each suggestion becomes a dropdown option
  ))}
</datalist>

```

How `<datalist>` works:

1. The `<input>` element has a `list` attribute pointing to the `<datalist>` by `id`.
2. The browser natively renders a dropdown below the input showing matching `<option>` values.
3. The user can click a suggestion or use arrow keys to select one.
4. When a suggestion is selected, the input's value is updated, which triggers `onChange`, which updates `query` state, which triggers the `useEffect`, which may fetch new suggestions.

`autoComplete="off"` disables the browser's built-in form autocomplete (which would show previously submitted values from browser history), so only our custom suggestions appear.

5. Complete data flow timeline

User types "p"

- query state = "p"
- `useEffect` fires: `query.length < 2`, clear suggestions, done

User types "pu" (50ms later)

- query state = "pu"
- `useEffect` fires: `clearTimeout` (nothing to clear), schedule timer for 300ms

User types "pum" (80ms later, before 300ms)

- query state = "pum"
- `useEffect` fires: `clearTimeout` (cancels the "pu" timer), schedule new timer for 300ms

User pauses (300ms passes with no more typing)

- timer fires: `getSuggestions("pum")` → API call
- API returns: `["pump", "pumping", "pump system"]`
- `setSuggestions(["pump", "pumping", "pump system"])`

- `datalist` re-renders with 3 options
- browser shows dropdown below input

Total API calls: 1 (not 3). The debouncing saved two unnecessary requests.

How to explain this in an interview

"We implement typeahead autocomplete with three components: a backend `/search/suggest` endpoint that queries JSONB metadata arrays using PostgreSQL's `jsonb_array_elements_text` to extract and match keywords with `ILIKE`; a debounced frontend effect that waits 300ms after the user stops typing before making the API call, using `useRef` for the timer and `useEffect` cleanup to cancel stale requests; and the native HTML `<datalist>` element that provides the dropdown UI with zero custom CSS. The suggestions come from AI-enriched keywords stored in category metadata, so they reflect actual searchable vocabulary in the database. The debounce prevents unnecessary API calls during continuous typing -- typically reducing requests by 80-90% compared to firing on every keystroke."